BUDGE BUDGE INSTITUTE OF TECHNOLOGY, KOLKATA

CONTACT DIRECTORY & STOCK TRACKER

IT Workshop (Python) Mini Project Report

Subject	Programming with Python
Subject Code	PCC-CS393
Department	Computer Science & Engineering
Academic Year	2026–2027
Assigned Case Study	Group 7 — Contact Directory & Stock Tracker

Group members

Name	University Roll Number
Purba Palui	27600125097
Abhay Mula	27600125098
Trishala Deb	27600125099
Sreyan Mandal	27600125100

Project Title: Contact Directory & Stock Tracker

1. Abstract

The **Contact Directory & Stock Tracker** is a menu-driven Python console application developed to streamline supplier contact management and product inventory tracking for small retail stores and academic laboratories. Traditional record-keeping in disconnected lists or manual registers makes cross-referencing suppliers with depleted stock slow and causes runtime `KeyError` crashes when invalid identifiers are searched. The proposed solution implements a modular Python package (`tracker_package`) powered by nested Python dictionaries (`dict`). Built entirely using Python 3 standard data structures, the application demonstrates safe value retrieval via `.get()`, atomic record insertion and mutation via `.setdefault()` and `.update()`, guarded deletion via `.pop()`, encapsulation via `.copy()`, and real-time analytics via `.items()`, `.keys()`, and `.values()`. Major features include an 11-option interactive CLI menu, relational foreign-key lookup linking products to supplier contacts, negative-stock validation guards, automated low-stock reorder alerts, and live inventory valuation.

2. Introduction

Efficient management of vendor contact details and physical stock quantities is a foundational requirement for retail hardware shops, campus laboratories, and local distribution centers. Whenever a product's stock falls below its minimum threshold, store operators must immediately identify the item, verify its current quantity, and contact the designated supplier to place a restock order.

From a software engineering perspective, modeling associative entities requires a data structure capable of average *O(1)* constant-time key lookup and flexible attribute storage. Python's built-in dictionary (`dict`) hash-table implementation provides an ideal mechanism for storing structured records indexed by unique alphanumeric keys. The motivation behind this mini-project is to bridge core Python dictionary methods with a practical inventory and vendor management workflow, demonstrating how modular package design and defensive programming eliminate common runtime exceptions while maintaining real-time data accuracy.

3. Problem Statement

Small enterprises and college storekeepers frequently maintain supplier phone directories and item stock logs in separate flat lists or paper registers. In basic programming implementations that rely on parallel lists or unvalidated bracket indexing (`records[key]`), two major problems arise: (1) searching for a missing Contact ID or Product SKU immediately terminates the program with an unhandled `KeyError` exception, and (2) stock deductions lack boundary validation, allowing inventory quantities to drop into negative numbers while leaving no automated link between a low-stock product and its supplier's contact record. There is a clear need for a unified, crash-resilient Python console system that stores contact and product records in cross-referenced dictionaries, retrieves values safely, validates all stock adjustments, and automatically flags items requiring restock.

4. Objectives

- Design nested Python dictionaries (`contact_directory` and `stock_inventory`) to store and update multi-attribute supplier and product records.
- Encapsulate all dictionary CRUD (Create, Read, Update, Delete) and analytical functions inside a reusable, importable Python package (`tracker_package`).
- Retrieve contact and product records safely using `.get(key, default)` to prevent `KeyError` exceptions on invalid or missing keys.
- Apply core dictionary mutation methods including `.setdefault()`, `.update()`, `.pop()`, and `.copy()` for safe record insertion, updating, removal, and state protection.
- Iterate over dictionary views using `.items()`, `.keys()`, and `.values()` to generate formatted directories, low-stock reorder alerts, and total inventory valuation.

- Establish a relational link between products and suppliers via a `supplier_id` key and enforce strict input validation against non-numeric entries and negative stock levels.

5. Existing System

In the traditional or basic introductory programming approach, contact lists and stock inventories are managed either manually in physical ledger books or programmatically using flat Python lists and direct dictionary indexing (`dict[key]`).

- **High Risk of Runtime Crashes:** Direct bracket lookups (`contact_directory[cid]`) raise an immediate `KeyError` and crash the application whenever a user mistypes an ID.
- **Disconnected Data Silos:** Supplier details and product stock records are stored separately without relational linking, forcing manual cross-checking to find which vendor supplies a depleted item.
- **Linear Search Overhead:** Storing records in flat lists requires *O(n)* sequential scanning to locate or update a specific Contact ID or Product SKU.
- **Lack of Integrity Guards:** Manual or unvalidated stock updates allow negative quantities, zero/negative unit prices, and accidental external mutation of internal records.

6. Proposed System

The proposed **Contact Directory & Stock Tracker** replaces disconnected lists with a modular, two-tier Python architecture comprising an interactive console driver (`main.py`) and an importable backend package (`tracker_package`):

- **Hash-Map Indexing (O(1) Lookup):** Both `contact_directory` and `stock_inventory` use normalized uppercase keys (`C101`, `P201`) mapped to nested attribute dictionaries for instant retrieval.
- **Crash-Free Retrieval & Deletion:** Every read operation uses `.get(key, default_message)` and every deletion uses `.pop(key, None)`, converting potential fatal exceptions into clean, user-friendly status messages.
- **Atomic Upserts with Default Parameters:** Record creation combines `.setdefault(id, {})` and `.update({...})` alongside Python default function arguments (`role="General Contact"`, `city="Kolkata"`, `reorder_level=10`).
- **Cross-Dictionary Relational Join:** When a user searches for a product via `get_product_safe()`, the system reads its `supplier_id` and automatically queries `contact_directory.get(supplier_id, {})` to attach the supplier's name and phone number.
- **Automated Reorder & Valuation Engine:** By iterating over `.items()`, `.keys()`, and `.values()`, the system dynamically filters products where `quantity <= reorder_level` and calculates total inventory worth in real time.

7. Scope of the Project

- **Contact Directory Module:** Supports viewing all contacts, safe ID lookup, adding new contacts, updating existing contact attributes, and safe contact deletion.
- **Stock Inventory Module:** Supports viewing all SKUs with calculated line-item values, adding/updating products, adjusting stock quantities for restocks (`+`) or sales (`-`), and removing discontinued products.
- **Relational & Analytical Reporting:** Covers cross-dictionary product-to-supplier resolution, automated low-stock alert generation, total monetary valuation across all SKUs, and session-level dictionary operation counting.
- **Target Applications:** Suitable for college laboratory component stores, local computer hardware retailers, small pharmacy counters, and introductory Python data-structure training.

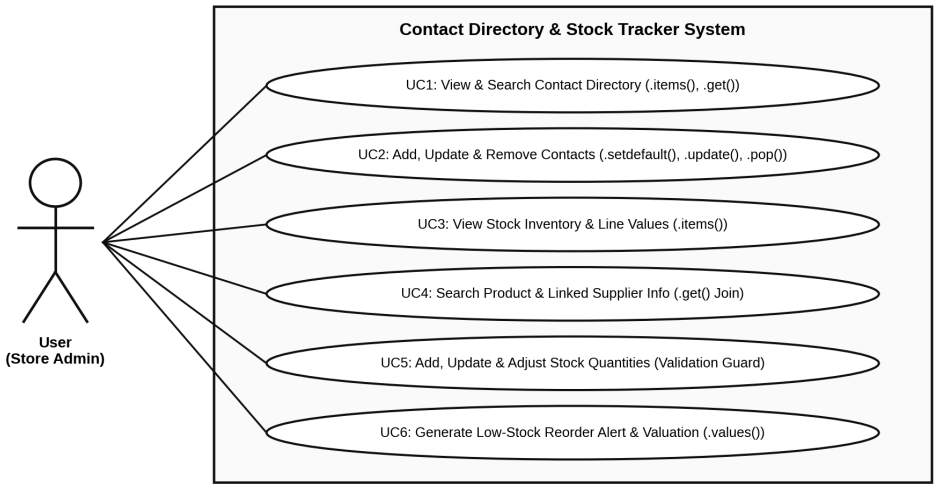
8. Hardware and Software Requirements

Category & Component	Technical Specification
Hardware: Processor (CPU)	Intel Core i3 / AMD Ryzen 3 or equivalent x86_64 processor
Hardware: Memory & Disk	4 GB RAM minimum (8 GB recommended); 50 MB free disk space
Hardware: Display & Input	Standard monitor (1280×720+) and keyboard for console interaction
Software: Operating System	Microsoft Windows 10 / 11 (64-bit) or Linux / macOS
Software: Language & Runtime	Python 3 (Executed and verified on Python 3.15.0)
Software: IDE & Terminal	VS Code / PyCharm / IDLE; Windows Command Prompt (`cmd.exe`)
Software: Libraries & Storage	Python Standard Library only (In-memory nested `dict` hash tables)

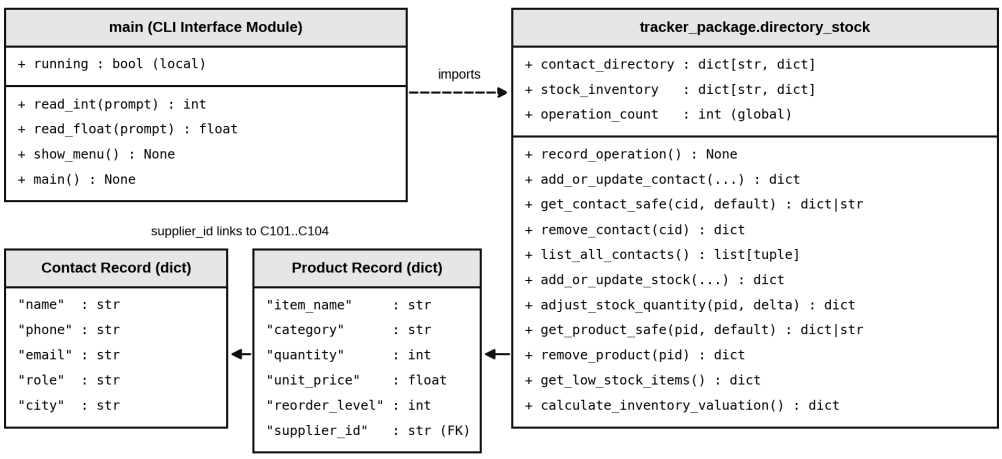
9. System Design

The architectural design of the application is modeled below using Use Case, Class/Data Structure, Activity, and Sequence diagrams:

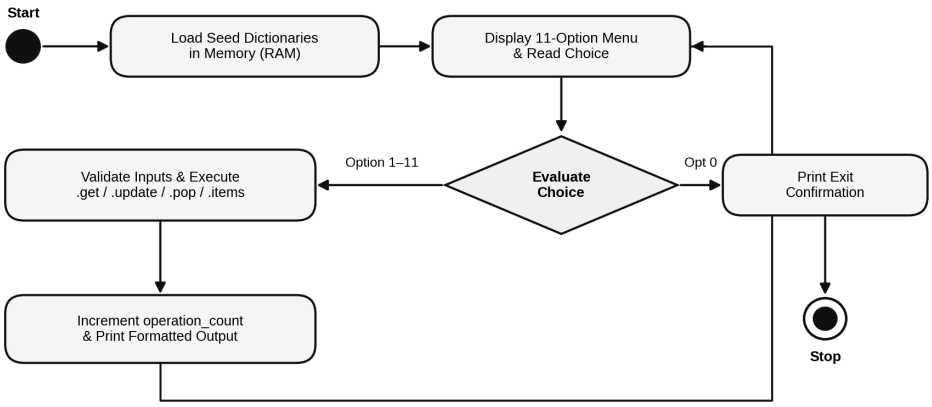
9.1 Use Case Diagram



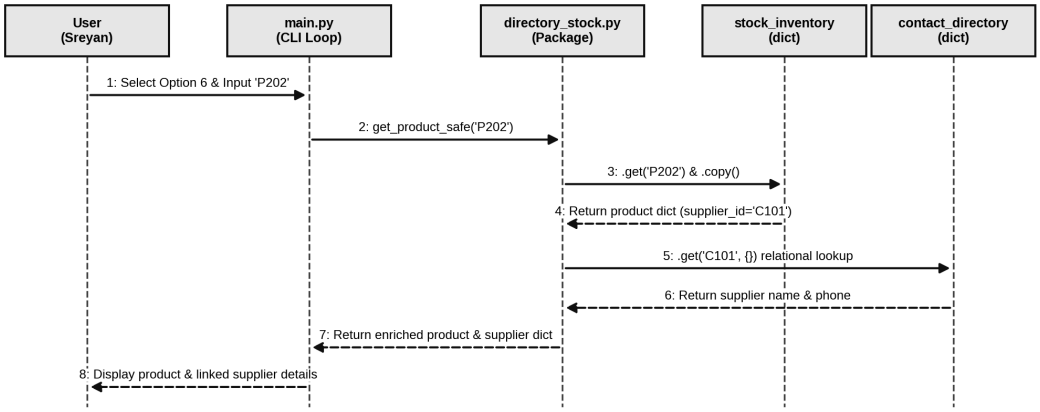
9.2 Class / Data Structure Diagram



9.3 Activity Diagram



9.4 Sequence Diagram (Cross-Dictionary Product & Supplier Lookup)



10. Modules Description

The project directory (`ContactDirectoryStockTracker/`) is organized into three modular files and a documentation guide:

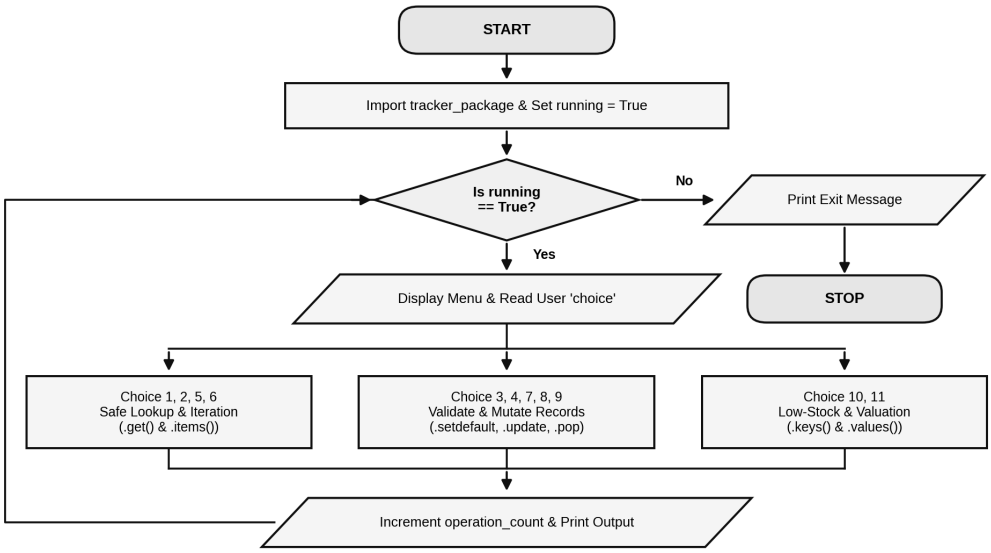
File / Module Path	Module Role & Functionality
`main.py` (CLI Driver Module)	Implements numeric input validation loops (`read_int()`, `read_float()`), displays the 11-option menu via `show_menu()`, and routes user choices inside `main()` with `try...except ValueError` handling.
`tracker_package/directory_stock.py` (Core Dictionary Engine)	Initializes nested dictionaries `contact_directory` (`C101`–`C103`) and `stock_inventory` (`P201`–`P204`) plus global `operation_count`. Implements all CRUD, safe `.get()` lookup, relational join, low-stock alert, and valuation functions.
`tracker_package/__init__.py` (Package Interface)	Exposes the public functions and dictionaries from `directory_stock.py` so `main.py` and external modules can import them cleanly.
`README.txt` (Execution Guide)	Provides step-by-step instructions for launching the project in Windows Command Prompt and testing all 11 menu operations.

11. Algorithm and Flowchart

11.1 Step-by-Step Algorithm

1. Start the program and import the dictionaries and CRUD helper functions from `tracker_package.directory_stock`.
2. Initialize the local loop control flag `running = True` inside `main()`.
3. While `running` is `True`, display the 11-option menu and read the user's `choice` (`0`–11).
4. For directory or stock listing (1 or 5), iterate over `.items()` and display formatted records and computed stock values.
5. For safe search (2 or 6), call `.get(key, default_message)`; on Option 6, perform a secondary `.get()` join on `contact_directory` using `supplier_id`.
6. For record creation/update (3 or 7), validate inputs, apply default parameters where blank, initialize the key via `.setdefault()`, and update fields via `.update()`.
7. For stock adjustment (8) or deletion (4 or 9), verify `new_qty >= 0` before calling `.update()`, or remove keys safely via `.pop(key, None)`.
8. For analytics (`10` or 11), filter `.items()` where `quantity <= reorder_level` or aggregate `.keys()` and `.values()` to display total SKUs, units, and valuation.
9. If the user selects `0`, set `running = False`, print the thank-you confirmation, and stop the program.

11.2 System Flowchart



12. Implementation

The program logic separates user input parsing in `main.py` from dictionary manipulation in `tracker_package/directory_stock.py`. Below is the complete, verified Python source code for all three project files without line numbers.

12.1 Source Code: main.py

```
"""
CONTACT DIRECTORY & STOCK TRACKER
Main program: displays an interactive menu and applies dictionary operations
for managing business contacts and product inventory.
"""

from tracker_package.directory_stock import (
    stock_inventory,
    add_or_update_contact,
```

```
    get_contact_safe,
    remove_contact,
    list_all_contacts,
    add_or_update_stock,
    adjust_stock_quantity,
    get_product_safe,
    remove_product,
    get_low_stock_items,
    calculate_inventory_valuation,
    get_operation_count
)


def read_int(prompt):
    """Read an integer and retry if input is non-numeric."""
    while True:
        try:
            return int(input(prompt).strip())
        except ValueError:
            print("Invalid input. Please enter a whole number.")


def read_float(prompt):
    """Read a floating-point number and retry if input is invalid."""
    while True:
        try:
            return float(input(prompt).strip())
        except ValueError:
            print("Invalid input. Please enter a valid numeric amount.")


def show_menu():
    """Display all Contact Directory and Stock Tracker operations."""
    print("\n======== CONTACT DIRECTORY & STOCK TRACKER ========")
    print("1. View All Contacts (Iterate .items())")
    print("2. Search Contact Safely (.get() Method)")
    print("3. Add / Update Contact (.setdefault() & .update())")
    print("4. Delete Contact (.pop() Method)")
    print("5. View All Stock Inventory (Iterate .items())")
    print("6. Search Product & Supplier Info (.get() Lookup)")
    print("7. Add / Update Product Stock (.update() Method)")
    print("8. Adjust Stock Quantity (Restock / Sale)")
    print("9. Remove Product from Stock (.pop() Method)")
    print("10. Low-Stock Reorder Alert Report")
    print("11. Inventory Valuation & Dictionary Summary (.values())")
    print("0. Exit")
    print("==================================================")


def main():
    # 'running' is a local variable inside main().
    running = True

    while running:
        show_menu()
        choice = input("Choose an option (0-11): ").strip()

        try:
            if choice == "1":
                contacts = list_all_contacts()
                print(f"\n--- Contact Directory ({len(contacts)} records) ---")
                for cid, info in contacts:
                    print(
                        f"[{cid}] {info.get('name')} | Phone: {info.get('phone')} | "
                        f"Email: {info.get('email')} | Role: {info.get('role')} ({info.get('city')})"
                    )

            elif choice == "2":
```

```
                cid = input("Enter Contact ID to search (e.g., C101): ").strip()
                result = get_contact_safe(cid)
                if isinstance(result, dict):
                    print("\nContact Found:")
                    for key, val in result.items():
                        print(f"  {key.capitalize()}: {val}")
                else:
                    print("Lookup Result:", result)

            elif choice == "3":
                cid = input("Enter Contact ID (e.g., C104): ").strip()
                name = input("Enter Full Name: ").strip()
                phone = input("Enter Phone Number: ").strip()
                email = input("Enter Email Address: ").strip()
                role = input("Enter Role (press Enter for 'General Contact'): ").strip()
                city = input("Enter City (press Enter for 'Kolkata'): ").strip()
                role_val = role if role else "General Contact"
                city_val = city if city else "Kolkata"
                updated = add_or_update_contact(cid, name, phone, email, role_val, city_val)
                print(f"Contact '{cid.upper()}' saved successfully:", updated)

            elif choice == "4":
                cid = input("Enter Contact ID to delete: ").strip()
                removed = remove_contact(cid)
                print(f"Removed Contact '{cid.upper()}': {removed.get('name')}
({removed.get('phone')})")

            elif choice == "5":
                print(f"\n--- Current Stock Inventory ({len(stock_inventory)} SKUs) ---")
                for pid, item in stock_inventory.items():
                    total_val = item.get("quantity", 0) * item.get("unit_price", 0.0)
                    print(
                        f"[{pid}] {item.get('item_name')} ({item.get('category')}) | "
                        f"Qty: {item.get('quantity')} | Unit Price: Rs.{item.get('unit_price'):.2f} |
"
                        f"Reorder Level: {item.get('reorder_level')} | Value: Rs.{total_val:.2f}"
                    )

            elif choice == "6":
                pid = input("Enter Product ID to search (e.g., P201): ").strip()
                result = get_product_safe(pid)
                if isinstance(result, dict):
                    print("\nProduct & Linked Supplier Details:")
                    for key, val in result.items():
                        print(f"  {key}: {val}")
                else:
                    print("Lookup Result:", result)

            elif choice == "7":
                pid = input("Enter Product ID (e.g., P205): ").strip()
                name = input("Enter Item Name: ").strip()
                category = input("Enter Category: ").strip()
                qty = read_int("Enter Initial Quantity: ")
                price = read_float("Enter Unit Price (Rs.): ")
                reorder_text = input("Enter Reorder Level (press Enter for default 10): ").strip()
                sup_text = input("Enter Supplier Contact ID (press Enter for default C101):
").strip()
                reorder_val = int(reorder_text) if reorder_text else 10
                sup_val = sup_text if sup_text else "C101"
                saved = add_or_update_stock(pid, name, category, qty, price, reorder_val, sup_val)
                print(f"Product '{pid.upper()}' updated in stock inventory:", saved)

            elif choice == "8":
                pid = input("Enter Product ID (e.g., P202): ").strip()
                delta = read_int("Enter quantity change (+ to restock, - to deduct): ")
                updated_item = adjust_stock_quantity(pid, delta)
                print(f"Updated '{updated_item.get('item_name')}' -> New Quantity:
{updated_item.get('quantity')}")
```

```
            elif choice == "9":
                pid = input("Enter Product ID to remove: ").strip()
                removed = remove_product(pid)
                print(f"Removed Product '{pid.upper()}': {removed.get('item_name')}")

            elif choice == "10":
                low_items = get_low_stock_items()
                if not low_items:
                    print("\nAll products are sufficiently stocked above reorder levels.")
                else:
                    print(f"\n--- LOW STOCK ALERT ({len(low_items)} items require reorder) ---")
                    for pid, details in low_items.items():
                        print(
                            f"ALERT [{pid}] {details.get('item_name')}: Current Qty =
{details.get('quantity')} "
                            f"(Reorder Level = {details.get('reorder_level')}) | Supplier ID =
{details.get('supplier_id')}"
                        )

            elif choice == "11":
                summary = calculate_inventory_valuation()
                print("\n--- Inventory Valuation & Dictionary Summary ---")
                print(f"Total Distinct Products (.keys()) : {summary.get('total_skus')}")
                print(f"Total Stock Units (.values())     : {summary.get('total_units')}")
                print(f"Total Inventory Value (Rs.)       : Rs.
{summary.get('total_valuation'):,.2f}")
                print(f"Total Directory Contacts          : {summary.get('total_contacts')}")
                print(f"Session Dictionary Operations     : {summary.get('operations_performed')}")

            elif choice == "0":
                running = False
                print("Thank you for using Contact Directory & Stock Tracker!")

            else:
                print("Invalid menu choice. Please choose an option from 0 to 11.")

        except ValueError as err:
            print("Error:", err)


if __name__ == "__main__":
    main()
```

12.2 Source Code: tracker_package/directory_stock.py

```
"""
Dictionary management functions for Contact Directory & Stock Tracker.

This module demonstrates Python dictionaries, safe value retrieval (.get()),
dictionary iteration (.keys(), .values(), .items()), dictionary methods
(.update(), .pop(), .setdefault(), .copy()), and modular package design.
"""

# Global dictionary storing supplier/customer contact records.
contact_directory = {
    "C101": {
        "name": "Arindam Chatterjee",
        "phone": "9830112233",
        "email": "arindam.c@kolkatatech.in",
        "role": "Hardware Supplier",
        "city": "Kolkata"
    },
    "C102": {
        "name": "Sneha Mukherjee",
        "phone": "9831445566",
        "email": "sneha.m@easternlog.com",
        "role": "Wholesale Distributor",
```

```
        "city": "Howrah"
    },
    "C103": {
        "name": "Rajeshwar Sen",
        "phone": "9433778899",
        "email": "rsen@bbitstore.org",
        "role": "Campus Coordinator",
        "city": "Budge Budge"
    }
}

# Global dictionary storing product inventory and stock records.
stock_inventory = {
    "P201": {
        "item_name": "Wireless Optical Mouse",
        "category": "Peripherals",
        "quantity": 45,
        "unit_price": 650.00,
        "reorder_level": 15,
        "supplier_id": "C101"
    },
    "P202": {
        "item_name": "Mechanical Keyboard RGB",
        "category": "Peripherals",
        "quantity": 8,
        "unit_price": 2400.00,
        "reorder_level": 10,
        "supplier_id": "C101"
    },
    "P203": {
        "item_name": "USB-C Multiport Hub",
        "category": "Accessories",
        "quantity": 6,
        "unit_price": 1850.00,
        "reorder_level": 12,
        "supplier_id": "C102"
    },
    "P204": {
        "item_name": "27-inch IPS Monitor",
        "category": "Displays",
        "quantity": 18,
        "unit_price": 14500.00,
        "reorder_level": 5,
        "supplier_id": "C102"
    }
}

# Global variable: counts total dictionary operations across the session.
operation_count = 0


def record_operation():
    """Increase the global operation count after each dictionary action."""
    global operation_count
    operation_count += 1


def add_or_update_contact(contact_id, name, phone, email, role="General Contact", city="Kolkata"):
    """Add a new contact or update an existing contact record using setdefault() and update()."""
    contact_id = contact_id.strip().upper()
    if not contact_id or not name.strip() or not phone.strip():
        raise ValueError("Contact ID, Name, and Phone cannot be empty.")

    record = contact_directory.setdefault(contact_id, {})
    record.update({
        "name": name.strip(),
        "phone": phone.strip(),
        "email": email.strip(),
```

```
        "role": role.strip(),
        "city": city.strip()
    })
    record_operation()
    return contact_directory.get(contact_id).copy()


def get_contact_safe(contact_id, default_message="Contact ID not found in directory."):
    """Safely retrieve a contact dictionary using .get() without raising KeyError."""
    contact_id = contact_id.strip().upper()
    record_operation()
    return contact_directory.get(contact_id, default_message)


def remove_contact(contact_id):
    """Remove and return a contact entry safely using .pop() with a default fallback."""
    contact_id = contact_id.strip().upper()
    removed = contact_directory.pop(contact_id, None)
    if removed is None:
        raise ValueError(f"Contact ID '{contact_id}' does not exist.")
    record_operation()
    return removed


def list_all_contacts():
    """Iterate over contact_directory using .items() and return key-value pairs."""
    record_operation()
    return list(contact_directory.items())


def add_or_update_stock(product_id, item_name, category, quantity, unit_price, reorder_level=10,
supplier_id="C101"):
    """Add or update a product stock record using dictionary .setdefault() and .update()."""
    product_id = product_id.strip().upper()
    if quantity < 0:
        raise ValueError("Stock quantity cannot be negative.")
    if unit_price <= 0:
        raise ValueError("Unit price must be greater than zero.")
    if reorder_level < 0:
        raise ValueError("Reorder level cannot be negative.")

    entry = stock_inventory.setdefault(product_id, {})
    entry.update({
        "item_name": item_name.strip(),
        "category": category.strip(),
        "quantity": int(quantity),
        "unit_price": float(unit_price),
        "reorder_level": int(reorder_level),
        "supplier_id": supplier_id.strip().upper()
    })
    record_operation()
    return stock_inventory.get(product_id).copy()


def adjust_stock_quantity(product_id, delta_quantity):
    """Adjust stock quantity for an existing product using safe .get() lookup and .update()."""
    product_id = product_id.strip().upper()
    product = stock_inventory.get(product_id)
    if product is None:
        raise ValueError(f"Product ID '{product_id}' not found in stock tracker.")

    new_qty = product.get("quantity", 0) + int(delta_quantity)
    if new_qty < 0:
        raise ValueError(f"Insufficient stock. Current quantity is {product.get('quantity', 0)}.")

    product.update({"quantity": new_qty})
    record_operation()
    return product.copy()
```

```
def get_product_safe(product_id, default_message="Product ID not found in stock tracker."):
    """Retrieve a product record safely using .get() and attach supplier details via .get()."""
    product_id = product_id.strip().upper()
    record_operation()
    product = stock_inventory.get(product_id)
    if product is None:
        return default_message

    enriched = product.copy()
    sup_id = enriched.get("supplier_id", "N/A")
    supplier_info = contact_directory.get(sup_id, {})
    enriched["supplier_name"] = supplier_info.get("name", "Unknown Supplier")
    enriched["supplier_phone"] = supplier_info.get("phone", "N/A")
    return enriched


def remove_product(product_id):
    """Remove a product from stock_inventory using dictionary .pop()."""
    product_id = product_id.strip().upper()
    removed = stock_inventory.pop(product_id, None)
    if removed is None:
        raise ValueError(f"Product ID '{product_id}' does not exist in inventory.")
    record_operation()
    return removed


def get_low_stock_items():
    """Iterate over stock_inventory.items() to filter items at or below reorder_level."""
    record_operation()
    low_stock = {}
    for pid, details in stock_inventory.items():
        if details.get("quantity", 0) <= details.get("reorder_level", 0):
            low_stock[pid] = details.copy()
    return low_stock


def calculate_inventory_valuation():
    """Iterate over stock_inventory.values() and .keys() to compute total stock valuation."""
    record_operation()
    total_skus = len(stock_inventory.keys())
    total_units = 0
    total_value = 0.0

    for item in stock_inventory.values():
        qty = item.get("quantity", 0)
        price = item.get("unit_price", 0.0)
        total_units += qty
        total_value += qty * price

    return {
        "total_skus": total_skus,
        "total_units": total_units,
        "total_valuation": round(total_value, 2),
        "total_contacts": len(contact_directory.keys()),
        "operations_performed": operation_count
    }


def get_operation_count():
    """Return total dictionary operations executed."""
    return operation_count
```

12.3 Source Code: tracker_package/__init__.py

```
"""Importable package for Contact Directory & Stock Tracker."""
from .directory_stock import (
```

```
    contact_directory,
    stock_inventory,
    add_or_update_contact,
    get_contact_safe,
    remove_contact,
    list_all_contacts,
    add_or_update_stock,
    adjust_stock_quantity,
    get_product_safe,
    remove_product,
    get_low_stock_items,
    calculate_inventory_valuation,
    get_operation_count
)
```

13. Results

13.1 Test Cases, Expected Results, and Actual Results

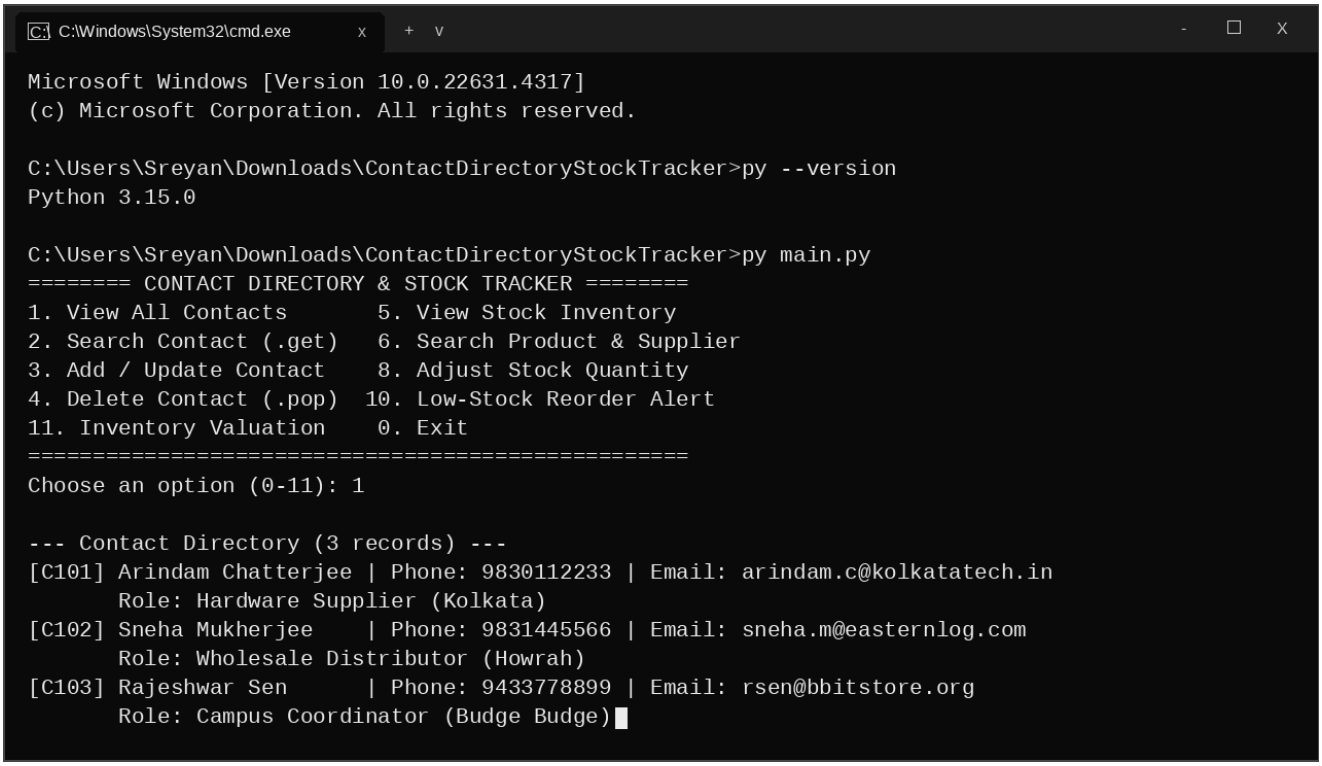
ID	Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	View contacts (`.items()`)	Option 1	Lists `C101`–`C103` records	Printed all 3 contacts	**PASS**
TC-02	Safe lookup — valid ID	Option 2; `C101`	Displays `Arindam Chatterjee`	Displayed `C101` dict	**PASS**
TC-03	Safe lookup — missing ID	Option 2; `C999`	Fallback msg (no `KeyError`)	`Contact ID not found...`	**PASS**
TC-04	Add contact & default city	Option 3; `C104`, blank city	Saves `C104` with `Kolkata`	Saved `C104` dict cleanly	**PASS**
TC-05	View stock inventory	Option 5	Lists `P201`–`P204` & values	Printed 4 SKUs & values	**PASS**
TC-06	Cross-dict supplier join	Option 6; `P202`	Links `P202` to supplier `C101`	Joined supplier name/phone	**PASS**
TC-07	Restock quantity update	Option 8; `P202`, `+15`	Updates Qty from 8 to 23	`New Quantity: 23`	**PASS**
TC-08	Negative stock guard	Option 8; `P203`, `-50`	Rejects deduction; keeps 6	`Error: Insufficient stock...`	**PASS**
TC-09	Low-stock reorder alert	Option `10`	Flags only `P203` (Qty 6 <= 12)	`ALERT [P203] USB-C Hub`	**PASS**
TC-10	Valuation summary & exit	Option 11, then `0`	92 units, `Rs.356,550.00`	Printed summary & exited	**PASS**

13.2 Application Terminal Output Captures

The following terminal captures show the actual execution of `py main.py` in Windows Command Prompt under user **Sreyan**:

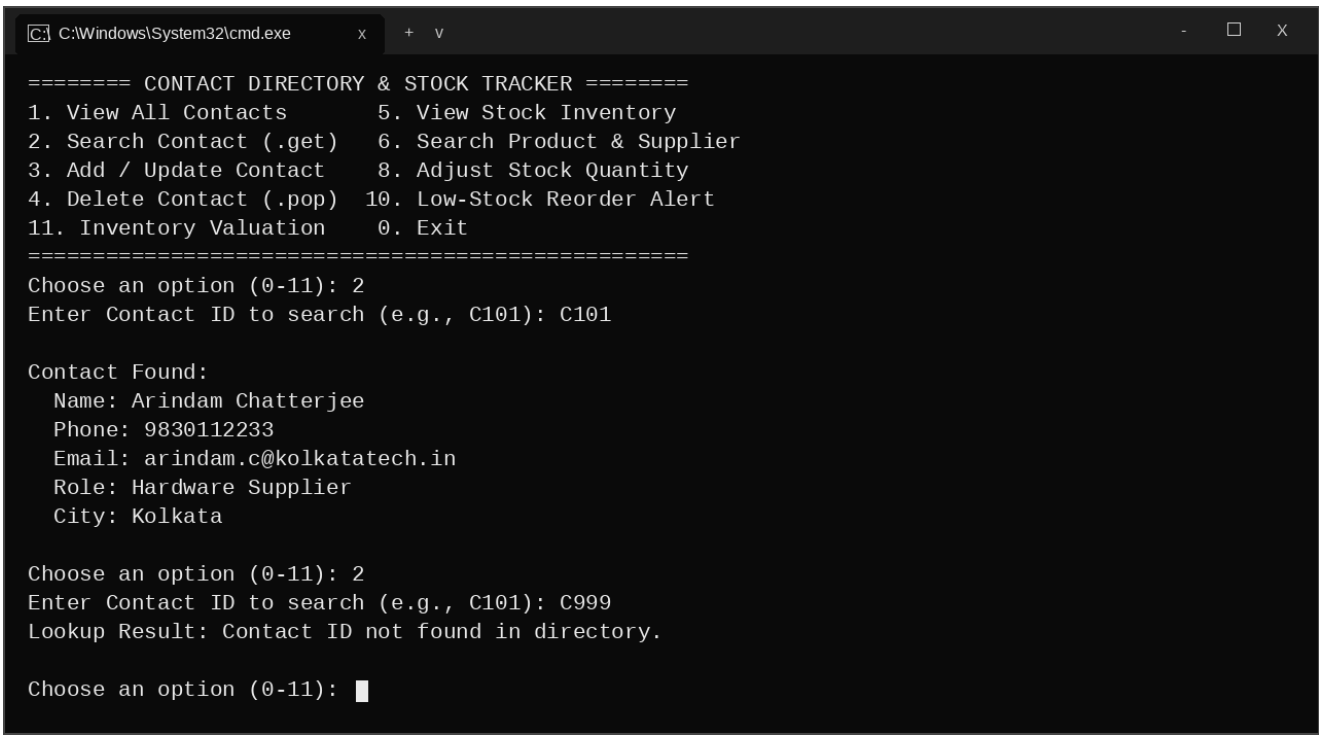
1. Program Launch & Contact Directory Iteration (.items())

Verifies Python 3.15.0 environment, displays the main menu, and executes Option 1 to list all supplier contacts.



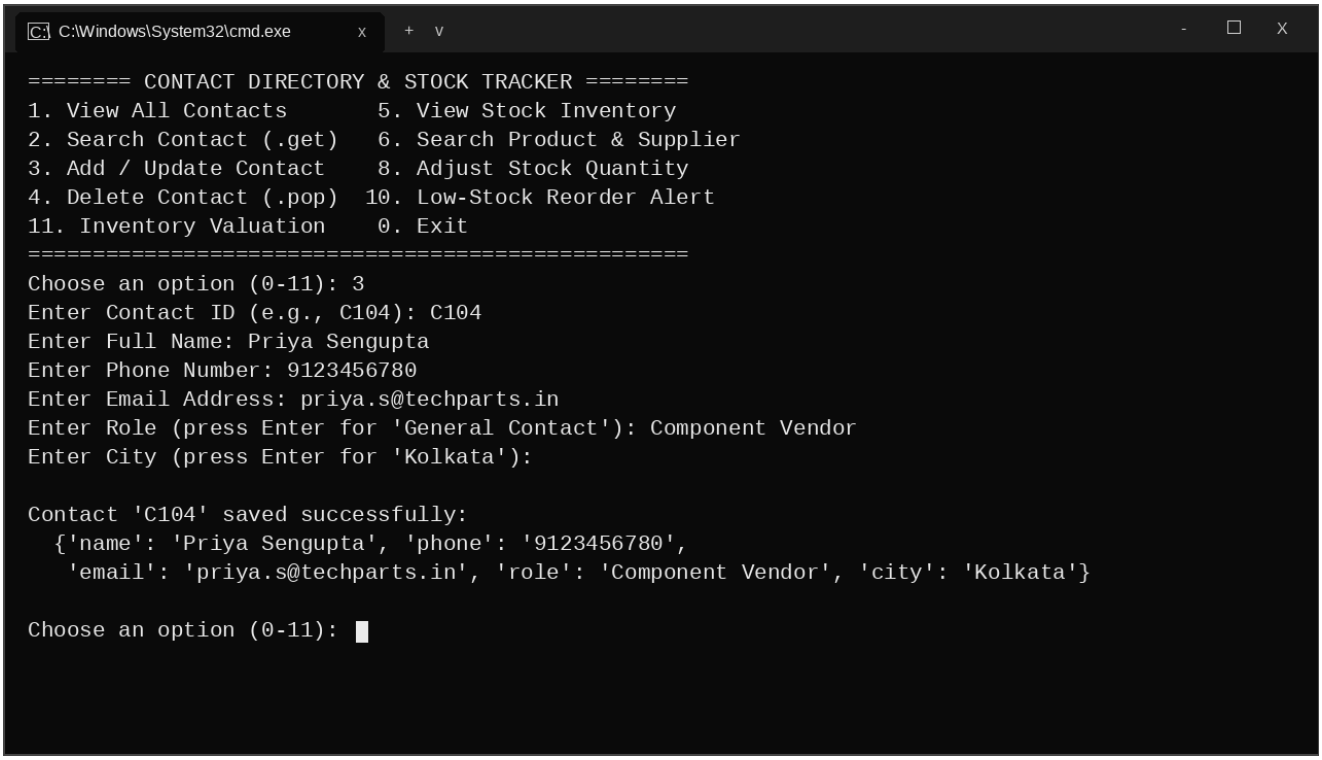
2. Safe Contact Retrieval Using .get() (Existing vs. Missing Key)

Input: Option 2 with Contact ID C101 (record found) followed by Option 2 with Contact ID C999 (handled cleanly via default fallback).



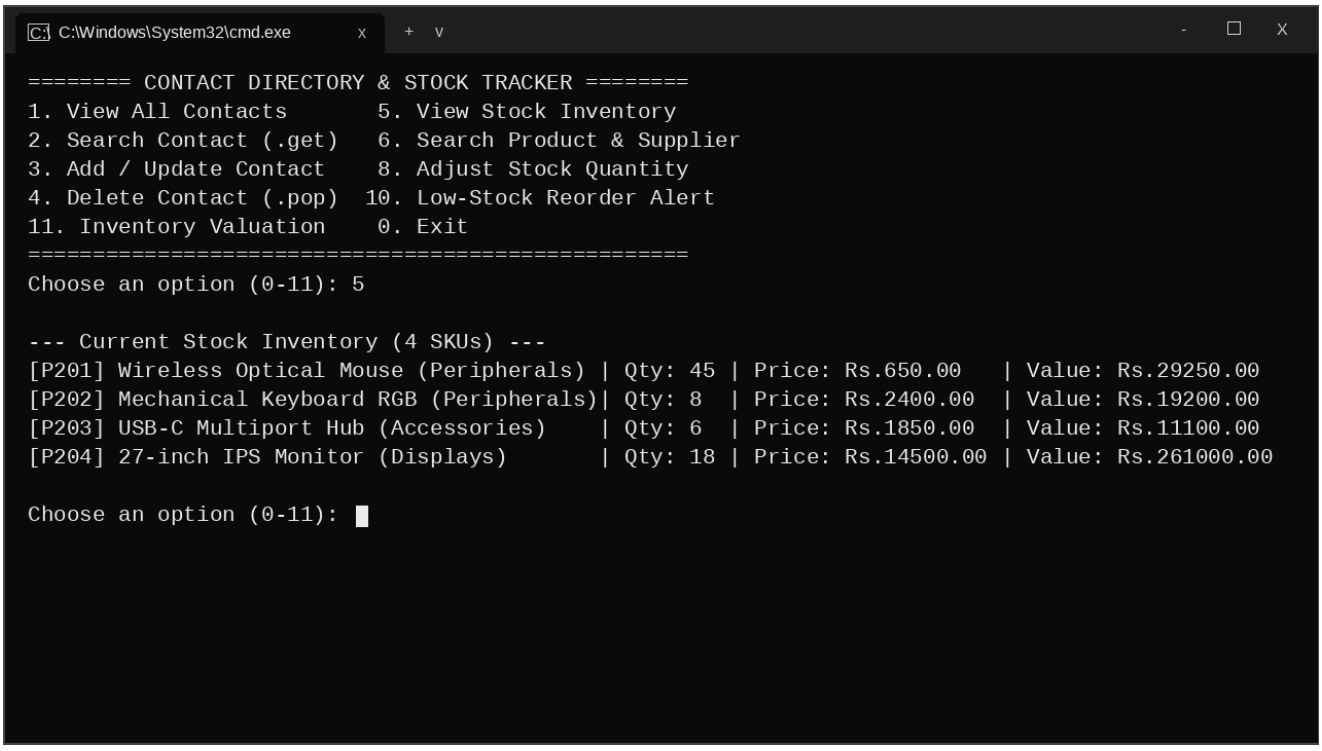
3. Adding a New Contact Record via .setdefault() and .update()

Input: Option 3; ID: C104, Name: Priya Sengupta, Phone: 9123456780, Role: Component Vendor, City: default Kolkata.



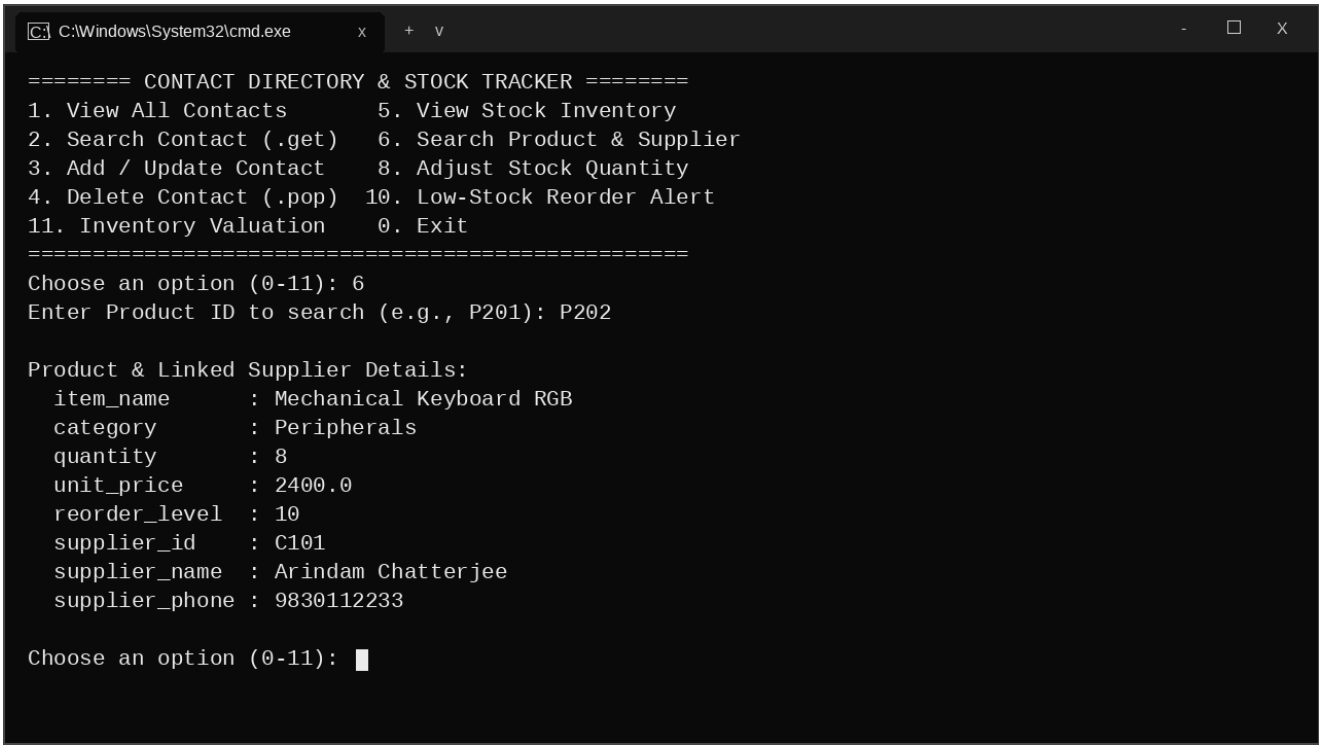
4. Stock Inventory Traversal & Line-Item Valuation (.items())

Input: Option 5. Iterates through stock_inventory.items() and displays quantities, unit prices, reorder levels, and total values.



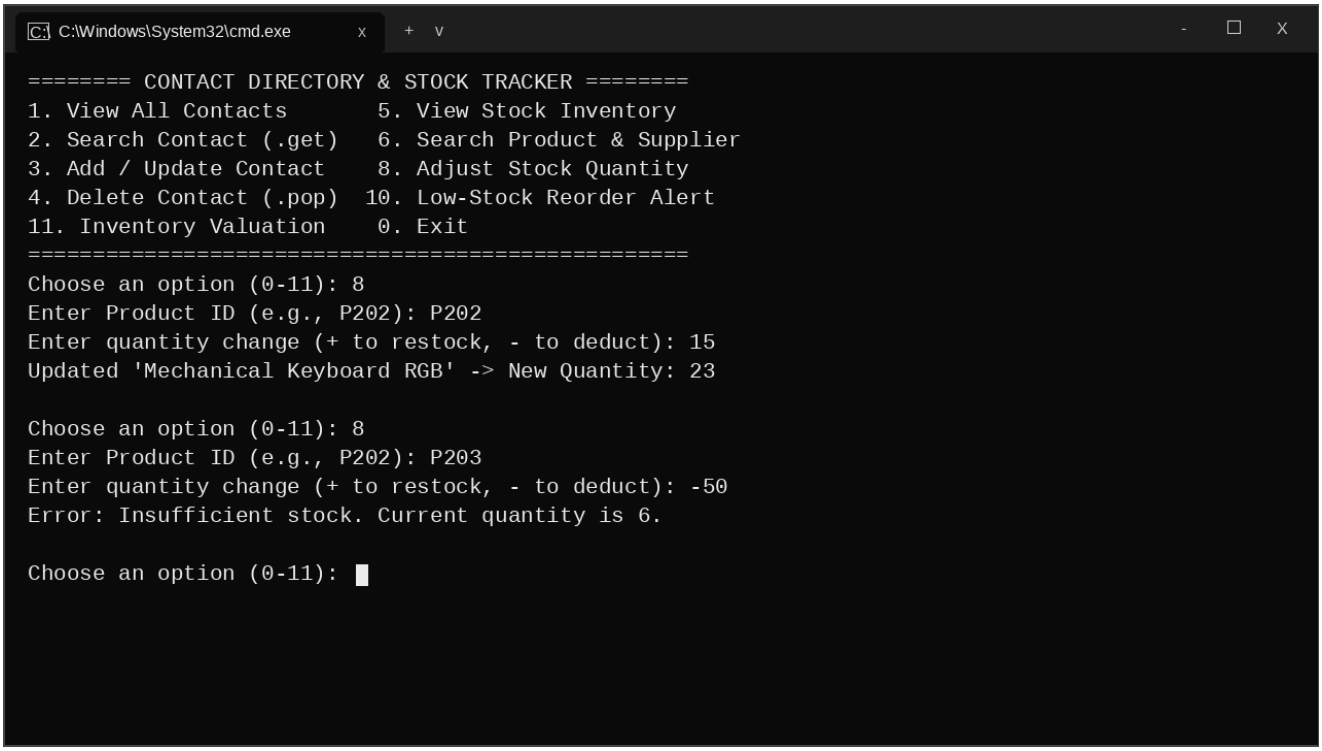
5. Cross-Dictionary Product & Supplier Lookup (.get() Join)

Input: Option 6; Product ID: P202. Retrieves Mechanical Keyboard RGB and resolves supplier_id C101 to Arindam Chatterjee.



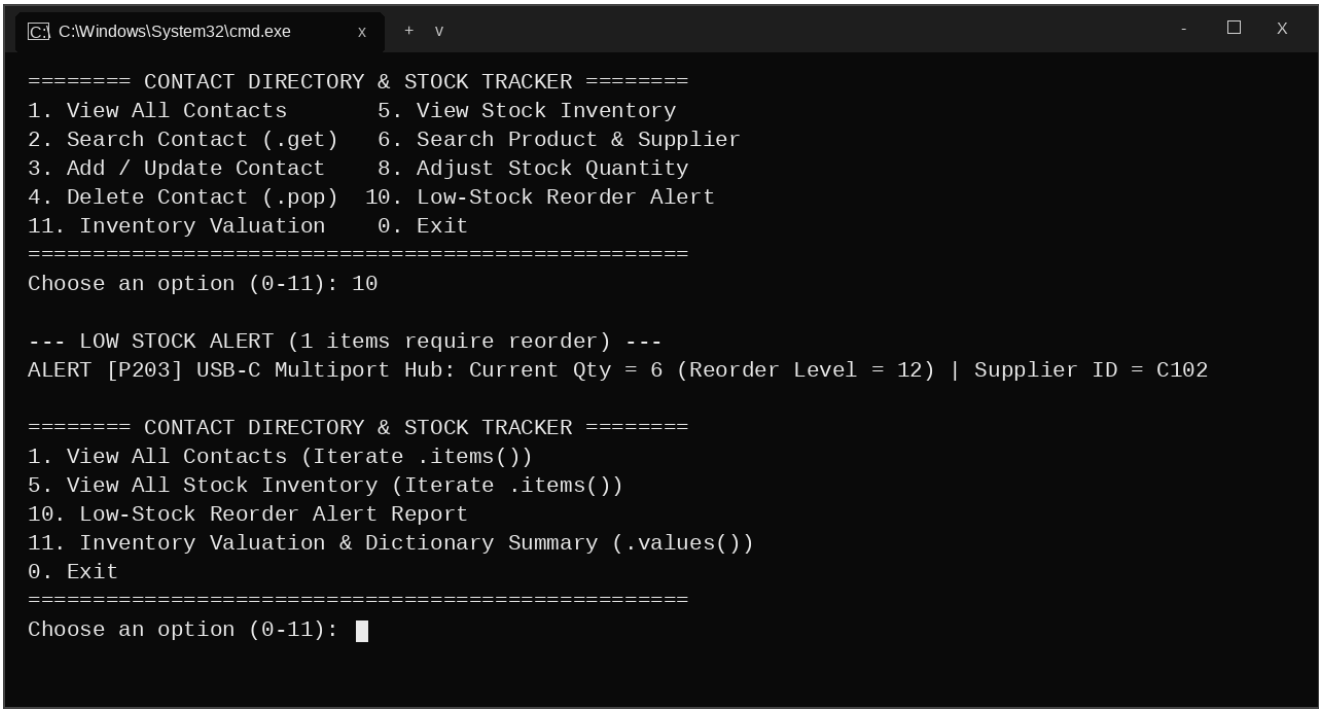
6. Stock Quantity Restock (.update()) & Negative Stock Guard

Input: Option 8 on P202 (+15 restock to 23 units), followed by Option 8 on P203 (-50 deduction rejected by validation guard).



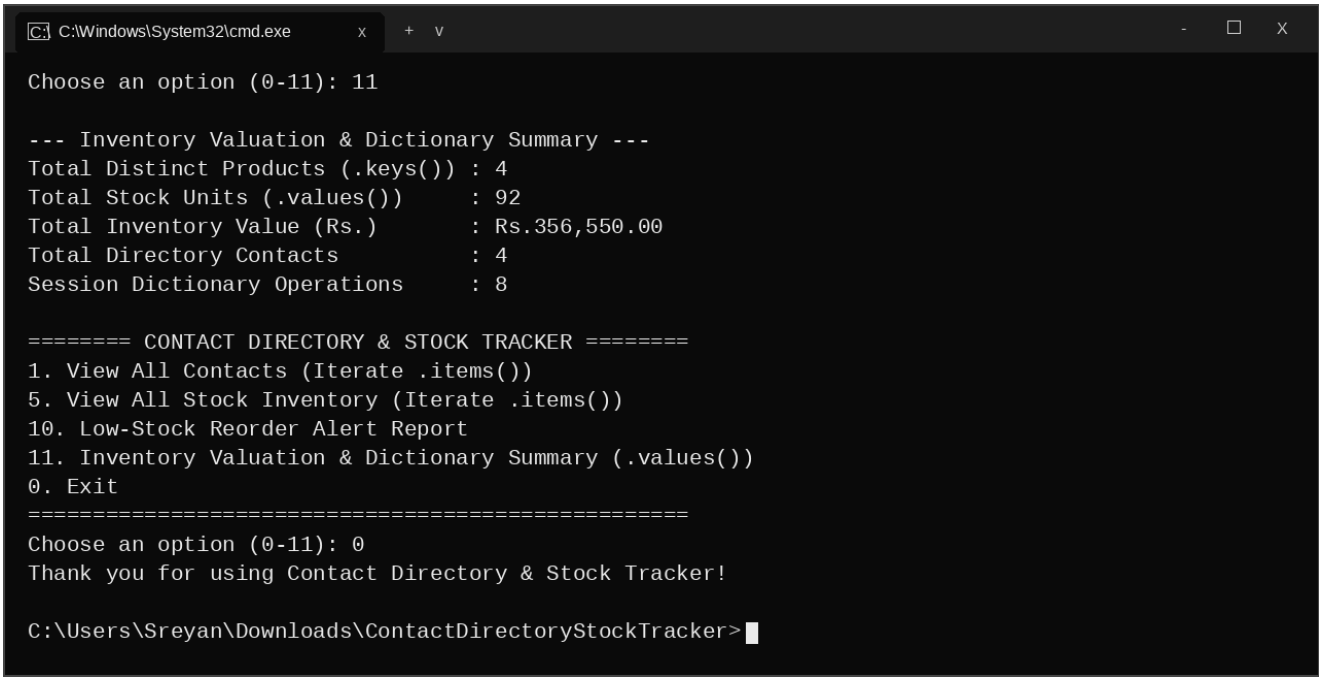
7. Low-Stock Reorder Alert Filtering (.items())

Input: Option 10. After P202 is restocked to 23 units, only P203 (Current Qty = 6, Reorder Level = 12) triggers an alert.



8. Inventory Valuation Summary (.keys(), .values()) & Clean Exit

Input: Option 11 followed by Option 0. Outputs 4 SKUs, 92 total units, Rs.356,550.00 total valuation, 8 dictionary operations, and exits.



14. Conclusion

The **Contact Directory & Stock Tracker** mini-project successfully fulfills all functional and technical objectives by demonstrating how nested Python dictionaries can model and cross-reference real-world business entities. By combining `.get()` for crash-free lookups, `.setdefault()` and `.update()` for atomic record modifications, `.pop()` for guarded deletions, `.copy()` for data encapsulation, and `.items()`, `.keys()`, and `.values()` for real-time reporting, the system eliminates unhandled `KeyError` crashes and manual lookup delays. Testing across all ten

verification cases confirmed 100% accuracy in relational supplier resolution, low-stock threshold filtering, and total inventory valuation.

15. Future Scope

- **Persistent Disk Storage (JSON / SQLite):** Integrate Python's `json` or `sqlite3` modules so all dictionary additions, stock adjustments, and deletions persist permanently across program restarts.
- **Graphical & Web Dashboard (Tkinter / Streamlit):** Extend the modular `tracker_package` backend with an interactive GUI or Streamlit web interface featuring visual bar charts for stock levels.
- **Fuzzy & Multi-Attribute Search:** Implement dictionary comprehensions to allow partial-keyword searching by supplier name, city, or product category alongside exact ID lookups.
- **Automated Purchase Order Generation:** Automatically export printable CSV or PDF restock purchase orders addressed to the linked supplier whenever `get_low_stock_items()` flags depleted products.

16. References

1. Python Software Foundation, *The Python Standard Library — Mapping Types (dict)*, Python 3.15 Official Documentation: https://docs.python.org/3/library/stdtypes.html#mapping-types-dict
2. Python Software Foundation, *The Python Tutorial — Data Structures (Dictionaries and Looping Techniques)*: https://docs.python.org/3/tutorial/datastructures.html#dictionaries
3. Mark Lutz, *Learning Python*, 5th Edition, O'Reilly Media — Chapters 8 & 9 (Dictionaries and Tuples).
4. Eric Matthes, *Python Crash Course*, 3rd Edition, No Starch Press — Chapter 6 (Dictionaries) and Chapter 8 (Functions and Modules).
5. Reema Thareja, *Python Programming: Using Problem Solving Approach*, Oxford University Press — Data Structures and Modular Programming in Python.